TradingQ Implementation Guide

Overview

This guide walks through implementing all recommendations for the TradingQ production deployment.

1. Application Code Implementation

Structure

```
terraform-sandbox/
├── app/
│   ├── main.py              # FastAPI application
│   ├── requirements.txt     # Python dependencies
│   └── Dockerfile           # Production Dockerfile
├── cicd/
│   └── github-actions.yml  # CI/CD pipeline
├── monitoring/
│   └── cloudwatch.tf       # Monitoring configuration
└── infra/
    └── envs/
        └── dev/
            └── terraform files
```

Application Features

- **FastAPI** REST API for trading order management
- **Health checks** at `/`, `/health`, `/ready`
- **Order management** endpoints for CRUD operations
- **Queue management** with status tracking
- **Background processing** for order execution simulation
- **Structured logging** with timestamps
- **Pydantic models** for validation

API Endpoints

Method	Endpoint	Description
GET	`/`	Root health check
GET	`/health`	Detailed health status
GET	`/ready`	Readiness probe
POST	`/api/v1/orders`	Create new order
GET	`/api/v1/orders/{id}`	Get order by ID
GET	`/api/v1/orders`	List orders (with filters)
DELETE	`/api/v1/orders/{id}`	Cancel order
GET	`/api/v1/queue/stats`	Queue statistics
GET	`/api/v1/queue`	Current queue contents

2. Build and Deploy Application

Step 1: Build the Application Image

```
cd ~/terraform-sandbox

# Copy application files
mkdir -p app
cp /home/claude/app/* app/

# Build Docker image
cd app
AWS_REGION=us-east-1
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
REPO="${ACCOUNT_ID}.dkr.ecr.${AWS_REGION}.amazonaws.com/tradingq-app"
TAG=$(date +%Y%m%d-%H%M%S)

# Login to ECR
aws ecr get-login-password --region $AWS_REGION | \
  docker login --username AWS --password-stdin "${ACCOUNT_ID}.dkr.ecr.${AWS_REGIO

# Build and push
```

```
docker build -t "tradingq-app:${TAG}" .
docker tag "tradingq-app:${TAG}" "${REPO}:${TAG}"
docker tag "tradingq-app:${TAG}" "${REPO}:latest"
docker push "${REPO}:${TAG}"
docker push "${REPO}:latest"

echo "Pushed image: ${REPO}:${TAG}"
```

Step 2: Test Locally (Optional)

```
# Run container locally
docker run -p 8080:8080 "tradingq-app:${TAG}"

# Test endpoints
curl http://localhost:8080/health
curl http://localhost:8080/api/v1/queue/stats

# Create test order
curl -X POST http://localhost:8080/api/v1/orders \
  -H "Content-Type: application/json" \
  -d '{
    "symbol": "AAPL",
    "order_type": "market",
    "side": "buy",
    "quantity": 100
  }'
```

Step 3: Deploy to ECS

```
cd ~/terraform-sandbox/infra/envs/dev

export TF_DATA_DIR=/tmp/tfdata-dev
export TF_PLUGIN_CACHE_DIR=/tmp/tfplugincache

terraform init -reconfigure

# Deploy with the new image
IMAGE="${REPO}:${TAG}"
terraform apply -auto-approve -var="image=${IMAGE}"
```

Step 4: Verify Deployment

```
# Get ALB DNS
```

```
ALB=$(terraform output -raw alb_dns_name)

# Wait for deployment
echo "Waiting for deployment..."
for i in {1..20}; do
  if curl -f "http://${ALB}/health" 2>/dev/null; then
    echo "✓ Deployment successful!"
    break
  fi
  echo "Attempt $i/20..."
  sleep 15
done

# Test API
curl "http://${ALB}/"
curl "http://${ALB}/health"
curl "http://${ALB}/api/v1/queue/stats"

# Create test order
curl -X POST "http://${ALB}/api/v1/orders" \
  -H "Content-Type: application/json" \
  -d '{
    "symbol": "TSLA",
    "order_type": "limit",
    "side": "buy",
    "quantity": 50,
    "price": 180.50
  }'

# List orders
curl "http://${ALB}/api/v1/orders"
```

3. Set Up CI/CD Pipeline

GitHub Actions Setup

1. **Create GitHub repository:**

```
cd ~/terraform-sandbox
git init
git remote add origin https://github.com/YOUR_USERNAME/tradingq.git
```

2. **Add GitHub Secrets:**
   - Go to Settings → Secrets → Actions

- Add:
  - `AWS_ACCESS_KEY_ID`
  - `AWS_SECRET_ACCESS_KEY`

3. **Copy workflow file:**

```
mkdir -p .github/workflows
cp /home/claude/cicd/github-actions.yml .github/workflows/deploy.yml
```

4. **Configure Terraform for GitHub:**

```
# Update infra/envs/dev/terraform.tfvars
# Or use terraform.tfvars.example as template
```

5. **Push to GitHub:**

```
git add .
git commit -m "Initial commit with application and CI/CD"
git push -u origin main
```

Pipeline Features

- ✅ Automated testing on PR
- ✅ Build and push Docker images
- ✅ Deploy to ECS on merge to main
- ✅ Deployment verification
- ✅ Staging environment support
- ✅ Rollback on failure

Manual Pipeline Trigger

If not using GitHub Actions, create a deployment script:

```
#!/bin/bash
# deploy.sh

set -euo pipefail

echo "Building and deploying TradingQ..."
```

```
# Build
cd app
TAG=$(date +%Y%m%d-%H%M%S)
REPO="${ACCOUNT_ID}.dkr.ecr.${AWS_REGION}.amazonaws.com/tradingq-app"

docker build -t "tradingq-app:${TAG}" .
docker tag "tradingq-app:${TAG}" "${REPO}:${TAG}"
docker push "${REPO}:${TAG}"

# Deploy
cd ../infra/envs/dev
terraform apply -auto-approve -var="image=${REPO}:${TAG}"

# Verify
ALB=$(terraform output -raw alb_dns_name)
curl -f "http://${ALB}/health" || exit 1

echo "✓ Deployment complete!"
```

4. Configure Monitoring and Alerting

Step 1: Add Monitoring to Terraform

```
cd ~/terraform-sandbox/infra/modules/app

# Add monitoring configuration
cp /home/claude/monitoring/cloudwatch.tf .
```

Step 2: Update Module Variables

Add to `variables.tf`:

```
variable "alert_email" {
  description = "Email for CloudWatch alerts"
  type        = string
}
```

Add to `envs/dev/terraform.tfvars`:

```
alert_email = "your-email@example.com"
```

Step 3: Deploy Monitoring

```
cd ~/terraform-sandbox/infra/envs/dev
terraform init
terraform apply -auto-approve -var="alert_email=your-email@example.com"
```

Step 4: Confirm SNS Subscription

Check your email and confirm the SNS subscription.

Step 5: Access CloudWatch Dashboard

```
# Get dashboard URL
terraform output dashboard_url

# Or navigate manually:
# AWS Console → CloudWatch → Dashboards → dev-tradingq-dashboard
```

Monitoring Features

Metrics Tracked:

- CPU and memory utilization
- API request count and error rates
- Response time/latency
- Unhealthy target count
- Error log patterns

Alarms Configured:

- High error rate (>10 errors in 5 min)
- High CPU (>80% for 10 min)
- High memory (>80% for 10 min)
- Unhealthy ALB targets
- High 5xx error rate
- Slow API responses (>2s average)

Dashboard Includes:

- Real-time resource utilization charts
- Request metrics and error rates
- API response time graph
- Recent error logs

5. Production Readiness Checklist

Security

- Secrets stored in AWS Secrets Manager
- IAM roles follow least privilege
- Security groups restrict access
- HTTPS/TLS enabled on ALB
- Container runs as non-root user

Reliability

- Health checks configured
- Auto-scaling policies set
- Multi-AZ deployment
- Deployment circuit breaker enabled
- Backup and disaster recovery plan

Monitoring

- CloudWatch alarms configured
- SNS notifications set up
- Dashboard created
- Log aggregation enabled
- Metrics collection active

Performance

- Load testing completed

- Resource limits tuned
- Database connection pooling
- Caching strategy implemented
- CDN for static assets

Operations

- CI/CD pipeline working
- Rollback procedure documented
- Incident response plan
- On-call rotation established
- Runbook created

6. Testing the Complete System

Functional Testing

```
ALB="your-alb-dns.us-east-1.elb.amazonaws.com"

# Health checks
curl "http://${ALB}/health"

# Create orders
for symbol in AAPL TSLA MSFT GOOGL; do
  curl -X POST "http://${ALB}/api/v1/orders" \
    -H "Content-Type: application/json" \
    -d "{
      \"symbol\": \"${symbol}\",
      \"order_type\": \"market\",
      \"side\": \"buy\",
      \"quantity\": 100
    }"
  sleep 1
done

# Check queue
curl "http://${ALB}/api/v1/queue/stats"
curl "http://${ALB}/api/v1/orders"

# Wait and verify processing
```

```
sleep 10
curl "http://${ALB}/api/v1/orders?status=completed"
```

Load Testing

```
# Install Apache Bench or use hey
go install github.com/rakyll/hey@latest

# Run load test
hey -n 1000 -c 10 -m GET "http://${ALB}/health"
hey -n 100 -c 5 -m POST -H "Content-Type: application/json" \
  -d '{"symbol":"AAPL","order_type":"market","side":"buy","quantity":10}' \
  "http://${ALB}/api/v1/orders"
```

Monitoring Test

```
# Trigger error alarm by generating errors
for i in {1..15}; do
  curl "http://${ALB}/api/v1/orders/nonexistent-id"
done

# Check if alarm fired (wait a few minutes)
aws cloudwatch describe-alarms \
  --alarm-names dev-tradingq-high-error-rate \
  --region us-east-1
```

7. Next Steps

Short Term

1. Add authentication/authorization
2. Implement rate limiting
3. Add request/response validation
4. Set up staging environment
5. Configure SSL/TLS certificates

Medium Term

1. Add database (RDS PostgreSQL)

2. Implement caching (ElastiCache Redis)
3. Add message queue (SQS) for order processing
4. Set up blue/green deployments
5. Implement API versioning

Long Term

1. Multi-region deployment
2. Integrate with actual trading APIs
3. Add WebSocket support for real-time updates
4. Implement advanced order types
5. Add machine learning for order optimization

8. Troubleshooting

See `TROUBLESHOOTING.md` for common issues and solutions.

Quick Checks

```
# Check service status
aws ecs describe-services \
  --cluster dev-tradingq-cluster \
  --services dev-tradingq-svc \
  --region us-east-1

# Check task status
aws ecs list-tasks \
  --cluster dev-tradingq-cluster \
  --service-name dev-tradingq-svc \
  --region us-east-1

# View logs
aws logs tail /ecs/dev-tradingq --follow --region us-east-1

# Check target health
TG_ARN=$(aws elbv2 describe-target-groups \
  --names dev-tradingq-tg \
  --query 'TargetGroups[0].TargetGroupArn' \
  --output text \
  --region us-east-1)
```

```
aws elbv2 describe-target-health \
  --target-group-arn "$TG_ARN" \
  --region us-east-1
```

Support

For issues or questions:

1. Check the troubleshooting guide
2. Review CloudWatch logs
3. Check GitHub issues
4. Contact the DevOps team

Last Updated: February 2026 **Version:** 1.0.0